RESEARCHEROS

StarterKIT

Первое исследование с AI-агентом

Прогноз взвешенной доли кликов на данных рекламы велосипедов

Восемь шагов: подключение проекта, дерево, карточка, постановка, запуск, результаты, редактура и два режима Arbor.

Читайте действия, копируйте готовое сообщение в указанный чат и проверяйте результат в конце каждого шага. Терминальные команды выполняются отдельно.

github.com/ZoyaV/researcheros_starter_kit

Набор данных: Sell me this! / 9 923 строки / 2023-2024

Версия руководства: 9 ноября 2026 г.

Содержание

1. Установка и создание дерева 2

2. Проверка дерева и первая карточка 4

3. Подробная постановка карточки 5

4. Запуск через /run 6

5. Монитор, результаты и выводы 7

6. Сокращение отчёта и словарь 8

7. Arbor из канбана: улучшение метода 9

8. Arbor из дерева: другие методы 10

До начала

Нужны Git, Git LFS, Python 3.10 или новее, ResearcherOS и настроенный CLI-агент. Сам эксперимент работает на CPU. Подписка или API выбранного агента нужны отдельно.

Ветка исходного набора - main. Дерево вы создадите сами; исследовательские материалы хранит ResearcherOS отдельно от кода.

Правила сравнения

Предсказываем числовой wCTR, не вероятность клика. Основная метрика - RMSE, меньше лучше. Постоянный прогноз сравнивается с деревом, затем с улучшенными вариантами.

Фиксированное разделение по датам: train - 6 093 строки, validation - 1 895, test - 1 935. Подбор выполняется на validation. Итоговый test открывается один раз в конце шага 8. Учебный порог улучшения - 1%; он не заменяет проверку статистической значимости.

Полное описание данных и протокола в репозитории.

1. Установка и создание дерева

Что попробуем

Подключим StarterKIT к ResearcherOS и создадим дерево исследования из описания задачи. Готовое дерево в набор не входит.

Действия

1. Установите Git и Git LFS. Установите и авторизуйте поддерживаемый агент командной строки (CLI), который хотите использовать в ResearcherOS. Подписка или API агента нужны отдельно; наличие приложения ResearcherOS их не заменяет.

2. Склонируйте StarterKIT. Команды ниже выполняются в терминале в папке, где вы храните проекты:

```
git clone --branch main https://github.com/ZoyaV/researcheros_starter_kit.git
cd researcheros_starter_kit
python scripts/check_data.py
```

На macOS / Linux, если команда `python` не найдена, используйте `python3`; на Windows можно использовать `py`. Дальше в руководстве `python` означает ваш Python 3.10 или новее.

3. Установите ResearcherOS из основной ветки. Команды выполняются в родительской папке, рядом со StarterKIT:

```
cd ..
git lfs install
git clone --branch main https://github.com/CognitiveAISystems/ResearcherOS.git ReseachOS
cd ReseachOS
git lfs pull
```

4. В папке `installers` запустите файл для вашей ОС: `Установить ResearcherOS.command` на macOS, `ResearcherOS Setup 0.1.0.exe` на Windows или `ResearcherOS-0.1.0.AppImage` на Linux. Для AppImage разрешите выполнение как программы. Установщик macOS собирает приложение с загрузкой зависимостей. Условия для Linux и обновления приведены в инструкции ResearcherOS.

5. Откройте приложение. При первом запуске на вопрос «Подключались ли вы с этого ноутбука к ResearcherOS?» выберите «Нет», затем «Выбрать репозиторий» и папку `researcheros_starter_kit`, не папку `data`. Если вы уже работаете в приложении, используйте создание нового дерева и выберите тот же репозиторий.

6. В мастере создания дерева вставьте описание из следующего блока. Ответьте на уточнения агента. Проверьте предложенное дерево и подтвердите кнопкой «Создать проект с этим деревом». Если мастер ещё не дошёл до этой кнопки, завершите его вопросы.

Как читать обозначения

Обучающая часть (`train`) нужна для обучения; проверочная (`validation`) - для выбора решений; итоговая тестовая (`test`) - для одной заключительной проверки. Основная метрика - корень из средней квадратичной ошибки (RMSE), меньше - лучше. Средняя абсолютная ошибка (MAE) помогает разобрать ошибки. Подробнее о выборках и признаках написано в `data/README.md`.

Сообщение для копирования

```
Хочу исследовать прогноз числовой взвешенной доли кликов (wCTR)
по данным рекламы велосипедов. Репозиторий researcheros_starter_kit
уже выбран. Прочитай AGENTS.md, data/README.md и data/protocol.json.
Одна строка - ячейка рекламного отчёта за день с признаками
объявления и аудитории. Большинство значений wCTR нулевые,
есть большие положительные значения. Это регрессия.
Помоги создать дерево: проблема -> проверяемые гипотезы ->
способы проверки или решения -> конкретные методы с канбаном.
Первый метод должен сравнить постоянный средний прогноз
с неглубоким деревом регрессии. Цель - проверить, есть ли
снижение RMSE хотя бы на 1% на фиксированной проверочной выборке.
Разделение по времени уже задано в data/protocol.json.
Не меняй протокол; итоговую тестовую выборку пока не оценивай.
Работаем локально на CPU, без GPU и облачных вычислений.
Сначала согласуем дерево, эксперименты пока не запускай.
```

Проверка результата

На главном экране есть ваше дерево. Внизу исследовательских ветвей находятся методы с канбаном. Приложение связано с локальной папкой StarterKIT. В дереве нет выдуманных результатов или отметок об успешных экспериментах.

2. Проверка дерева и первая карточка

Что попробуем

Проверим связь проблемы и методов, затем создадим первую карточку в канбане метода.

Действия

1. Выберите созданное дерево в боковой навигации. На полотне проследите путь до метода: проблема, проверяемая гипотеза, способ её проверки или решения, метод. Ветви исследования должны заканчиваться конкретными методами с канбаном, а не общей идеей «улучшить качество».

2. Откройте боковой чат кнопкой «Открыть чат». Если панель уже видна, используйте её. В списке чатов нажмите «Новый чат» и проверьте, что выбран именно ваш проект. Внизу выберите настроенный CLI-агент.

3. Отправьте сообщение ниже. Если агент задаёт вопрос, ответьте в этом же чате; не запускайте исследование параллельно в другом окне.

Сообщение для копирования

```
Проверь дерево этого StarterKIT: логично ли оно связывает
проблему прогноза wCTR с проверяемыми гипотезами и методами?
Убедись, что ветви заканчиваются методами с канбаном.
Посоветуй первый небольшой эксперимент для CPU:
постоянный средний прогноз против неглубокого дерева регрессии
по data/README.md и неизменному data/protocol.json.
Объясни, какую гипотезу проверяем и чего результат не докажет.
Найди подходящий метод. Если карточки такого опыта ещё нет,
создай одну карточку в backlog; если она уже есть, используй её.
Не создавай дубликаты и не запускай опыт.
Дай название метода и карточки и ссылку для открытия в приложении,
если можешь её сформировать. Иначе объясни, где их найти.
```

Проверка результата

У конкретного метода есть одна карточка первого опыта в колонке backlog. Агент объяснил, что именно проверяется. Если дерево заканчивается гипотезой без метода, сначала попросите добавить метод; не создавайте карточку без связи с ним.

3. Подробная постановка карточки

Что попробуем

Откроем карточку и договоримся с агентом о проверке до запуска кода.

Действия

1. Откройте карточку по ссылке из чата. Если ссылка не дана или не открылась, найдите названный метод на дереве, откройте его канбан и нажмите карточку в backlog.
2. В окне карточки откройте отчёт и чат. В этой версии окно называется «Отчёт»; краткое резюме карточки и полный отчёт - разные уровни описания. Для уточнения постановки нужен чат именно открытой карточки, а не общий чат без привязки.
3. Введите `/`, чтобы увидеть команды карточки. Отправьте блок ниже с `/grill-me`. Агент задаёт вопросы по одному и предлагает ответ. Подтвердите подходящие решения или исправьте их.

Сообщение для копирования

```
/grill-me Уточни постановку этой карточки по её краткому описанию.
Прочитай data/README.md и data/protocol.json. До запуска заполни
в отчёте цель, основную метрику, правило вывода и подготовку.
Первое сравнение: константа среднего train и дерево регрессии
max_depth=3, min_samples_leaf=20, random_state=42.
Главная метрика - RMSE; MAE и ошибки на нулевых/положительных
значениях используй только для диагностики.
Критерий: проверочная RMSE хотя бы на 1% ниже константы.
Признаки и временное разделение фиксированы; test пока не трогать.
Обработку категорий обучай только на train, неизвестные категории
обрабатывай корректно. Текст можно исключить в первом опыте,
но запиши это ограничение. Не используй row_id как признак.
Предусмотри локальное окружение, воспроизводимую команду,
чеклист подзадач, таблицу результатов и график ошибок для монитора.
Ограничь первый опыт 10 минутами на CPU, без GPU и облака.
Запуск - отдельным сообщением /run после окончания постановки.
```

Проверка результата

В отчёте есть цель, RMSE и заранее заданный порог, фиксированные выборки, описание моделей и обработки признаков, команда запуска, бюджет и подзадачи. Карточка ещё не выполняется. Чисел будущего качества в отчёте нет.

4. Запуск через /run

Что попробуем

Выполним согласованную карточку и увидим работу агента в проекте.

Действия

1. Завершите вопросы постановки из шага 3. В чате карточки проверьте выбранного CLI-агента. Если для установки зависимостей агенту нужен доступ, разрешите только необходимые действия в локальном окружении проекта.

2. Окружение можно создать самостоятельно из корня StarterKIT. Выполните команды в терминале:

```
python -m venv .venv
```

Активируйте его: `source .venv/bin/activate` на macOS / Linux или `.venv\Scripts\Activate.ps1` в Windows PowerShell. Затем:

```
python -m pip install -r requirements.txt
python scripts/check_data.py
```

3. Вставьте следующий блок в чат карточки. Повторно отправлять `/run`, пока первый запуск работает, не нужно.

Сообщение для копирования

```
/run Выполни согласованную постановку этой карточки.
Используй окружение .venv, фиксированные данные и протокол.
Создай минимальный воспроизводимый код первого эксперимента,
сравни константу и дерево на validation. Test не оценивай.
Сохрани команду, конфигурацию, версии библиотек, метрики,
предсказания и график ошибок в отдельную папку runs/.
Подключи реальные журнал и изображения к монитору карточки.
Отмечай подзадачи по мере выполнения и обновляй краткий статус.
После окончания запиши результаты, ограничения и вывод,
синхронизируй статус карточки. Слабое качество не скрывай.
Если запуск сломан или превышен бюджет 10 минут на CPU,
останови его, сохрани диагностику и не объявляй опыт успешным.
```

Проверка результата

Карточка переходит в работу. Появляются код и реальные артефакты в `runs/`. Если агент просит доступ или уточнение, ответьте в его чате. В случае ошибки сначала разберите её; не считайте отсутствие метрик отрицательным научным результатом.

5. Монитор, результаты и выводы

Что попробуем

Проследим выполнение и проверим, что вывод следует из измерений.

Действия

1. Пока карточка работает, найдите у метода значок лупы и откройте монитор. Смотрите краткий статус, подзадачи, журнал и графики. Если короткий опыт уже завершился, откройте его отчёт и сохранённые артефакты: пустой текущий монитор сам по себе не означает ошибку.

2. После завершения откройте карточку. В отчёте должна быть таблица RMSE константы и дерева на одной проверочной выборке, диагностические показатели и график ошибок. Сначала результаты, затем их интерпретация.

3. Посмотрите короткий вывод на канбане и сведения в базе знаний через кнопку Knowledge. Если вывод ещё не появился, попросите агента закончить оформление этой карточки, а не запускать новый опыт.

4. Отправьте проверочный вопрос в чате карточки. Команда `/question` предназначена для ответа по материалам без изменений.

Сообщение для копирования

```
/question Объясни результат этой карточки простым языком.
Покажи RMSE константы и дерева на validation и вычисли
относительное изменение: 100 * (RMSE_константы - RMSE_дерева)
/ RMSE_константы. Выполнен ли заранее заданный порог 1%?
Если RMSE константы нулевая, не дели на ноль.
Отдели ошибки на нулевых и положительных значениях wCTR.
Дай ссылки на таблицу, график, команду и сохранённые артефакты.
Что мы узнали и чего этот опыт не доказывает?
Test не вычисляй и не показывай. Ничего не изменяй.
```

Проверка результата

Числа ответа совпадают с отчётом и файлами. Вывод не объявляет дерево лучше, если порог не достигнут. Карточка может быть завершена с отрицательным результатом; ошибка исполнения должна быть описана как ошибка, а не как опровержение гипотезы.

6. Сокращение отчёта и словарь

Что попробуем

Сделаем отчёт удобнее для чтения и зададим предпочтительные термины для своего дерева.

Действия

1. Откройте завершённую карточку и её чат. Можно продолжить прежний разговор или через список чатов нажать «Новый чат карточки». Проверьте, что новый разговор связан с той же карточкой.

2. Отправьте первый блок с `/compress`. Эта команда вызывает скилл `koi-reader-rewrite`: он проверяет читаемость и при необходимости переписывает текст. Уже понятный отчёт может остаться без изменений.

3. Прочитайте сокращённую версию. Проверьте сохранность метрик, ограничений, правила вывода, таблиц и изображений.

4. Откройте «Словарь» в канбане метода. Добавьте правило `бейзлайн > исходное решение` и сохраните. Здесь правило действует для выбранного дерева. Не используйте «Словарь для всех деревьев», если не хотите менять терминологию других проектов.

5. После сохранения словаря отправьте второй блок в чат той же карточки. Вместо примера можно добавить свой термин в формате `термин > замена`.

Сообщение для сокращения

```
/compress Сделай отчёт этой карточки понятным исследователю,
который не видел наш запуск. Сохрани числа, критерий 1%,
таблицы, изображения, ограничения, ссылки и воспроизводимость.
Не меняй смысл отрицательного результата и не усиливай вывод.
Если текст уже читается хорошо, объясни, почему правка не нужна.
```

Сообщение после изменения словаря

```
Прочитай сохранённый словарь этого дерева заново.
Перепиши только прозу отчёта текущей карточки с учётом правила
«бейзлайн > исходное решение». Не меняй числа, код, имена файлов,
цитаты, идентификаторы, правило решения и статус карточки.
Если слово не встречается, не добавляй его искусственно.
Покажи, какие фрагменты изменились. Другие отчёты не редактируй.
```

Проверка результата

Отчёт сохранил факты и изображения. Замены применены к прозе текущего отчёта. Словарь сам по себе не переписал старые отчёты и не изменил другие деревья.

7. Arbor из канбана: улучшение метода

Что попробуем

Запустим ограниченное автоисследование внутри уже проверенного метода.

Действия

1. Дождитесь завершения первого опыта. Откройте канбан того же метода и чат метода или чат его карточки. Проверьте название выбранного метода перед отправкой.

2. Введите `/` и выберите «Автоисследование» или вставьте команду из блока ниже.

3. В окне «Масштаб автоисследования» выберите «В рамках метода». В этом упражнении не выбирайте «На уровне дерева».

4. Согласуйте бюджет: до 15 минут, не более двух экспериментальных запусков, один одновременно, только CPU. Если окно задаёт число исполнителей, ограничьте одновременно работающих исполнителей одним; текстовый предел двух запусков сохраняется. Не подтверждайте настройки, разрешающие больший бюджет.

Сообщение для копирования

```
/koi-arbor-autoresearch Улучши выбранный метод дерева регрессии
по итогам первой карточки. Масштаб - только этот метод.
Прочитай его отчёты и зафиксируй исходную проверочную RMSE.
Предложи проверяемые гипотезы улучшения, например другую глубину
или размер листа; не создавай конкурирующие методы в дереве.
Сначала опубликуй гипотезы и карточки опытов в канбане метода,
подготовь постановки, затем выполняй их по очереди.
Протокол, признаки и разделение не меняй. Test не оценивай.
Успех: RMSE validation хотя бы на 1% ниже лучшего ранее
проверенного решения; если оно всё ещё константа, скажи об этом.
Бюджет: до 15 минут и не более двух экспериментальных запусков,
один одновременно; только CPU, без GPU и облака.
Используй независимые копии, если нужна изоляция кода;
не создавай ветки и worktree в StarterKIT.
Обновляй монитор реальными графиками. По завершении сохрани
сравнение с прошлым результатом, выводы и причину остановки.
Не заявляй улучшение, если критерий не выполнен.
```

Проверка результата

Новые карточки и гипотезы находятся внутри выбранного метода. В отчётах сравниваются сопоставимые метрики, соблюдён бюджет. Если улучшения нет, это явно записано. Этот запуск не создаёт новые методы и не меняет корень дерева.

8. Arbor из дерева: другие методы

Что попробуем

Расширим поиск за пределы одного метода, затем один раз проверим выбранный результат на итоговой тестовой выборке.

Действия

1. Завершите предыдущий запуск Arbor. Закройте канбан и окно карточки, вернитесь на полотно дерева. Откройте чат дерева; убедитесь, что это не разговор карточки или метода.

2. Вызовите «Автоисследование» через меню / или отправьте первый блок ниже. В окне «Масштаб автоисследования» подтвердите исследование на уровне дерева кнопкой «Начать исследование». Выделенный на полотне метод не должен сужать этот запрос до его настройки.

3. Сохраните те же пределы: до 15 минут, не более двух экспериментальных запусков, один одновременно, только CPU. Дождитесь сравнения методов на validation и фиксации выбранного варианта.

4. После выбора победителя отправьте второй блок в том же чате. Это единственное обращение к итоговому тесту за весь маршрут. Если test уже использовали раньше, сообщите об этом: он больше не является нетронутой проверкой.

Сообщение для поиска методов

```
/koi-arbor-autoresearch Исследуй другие методы прогноза числового
wCTR на уровне этого дерева. Сначала прочитай полученные выводы.
Не ограничивайся настройкой прежнего дерева регрессии.
Предложи проверяемые гипотезы других решений, например модель
наличия отклика плюс регрессию положительного значения
или случайный лес с ограничением размера.
Публикуй методы в подходящих ветвях и готовь их карточки
до запуска. Корневую проблему не меняй.
Сравнивай с лучшим ранее проверенным решением по RMSE validation;
порог улучшения - 1%. Разделение и протокол не меняй.
Test пока не оценивай: сначала зафиксируй победителя по validation.
Бюджет: до 15 минут и двух экспериментальных запусков,
один одновременно, только CPU, без GPU и облака.
Не создавай ветки и worktree в StarterKIT.
Сохрани результаты, отрицательные исходы, ограничения и выводы.
Остановись по достижении цели или исчерпании бюджета.
```

Сообщение для итоговой проверки

```
Победитель уже выбран по validation. Зафиксируй его конфигурацию
и исходное решение до вычисления итоговых метрик.
Один раз оцени на test выбранного победителя, константу
и первый вариант дерева; не подбирай ничего по этим числам.
Используй прежние модели, обученные на train, без дообучения
на validation. Приведи RMSE, MAE и число строк для каждого.
Проверь, сохраняется ли снижение RMSE хотя бы на 1%
относительно зафиксированного исходного решения.
Сохрани таблицу, предсказания и график, затем вывод в дереве.
Если улучшение не перенеслось на поздние даты, прямо напиши это.
Повторный поиск по раскрытому test не запускай.
Заверши учебный маршрут и перечисли полученные артефакты.
```

Проверка результата

В дереве появились обоснованные альтернативные методы и ссылки на их эксперименты. Финальный отчёт отделяет выбор по validation от однократной проверки на test. В базе знаний есть выводы с основаниями, в том числе отрицательные. Новые поиски после просмотра test требуют нового протокола и новой нетронутой выборки.